

THEORY OF COMPUTATION

CAPSTONE ASSIGNMENT

NAME $\Rightarrow$ SNEHA

ROLL NO $\Rightarrow$ 2301010398

COURSE $\Rightarrow$ BTech CSE

SEC $\Rightarrow$ F

SUBMITTED TO :

MANSI KASAL

Ma'am

Unit 1 - Finite Automata and Regular Expression

To match valid identifiers (starting with alphabets followed by alphabets (digits) & exclude the keywords for, while, if the regular expression is

Regular Expression for valid identifiers
 Let ΣA be the set of alphabets & ΣD be the set of digits. The regular expression R for all valid identifiers (alphabet followed by any sequence of alphabets or digits) is

$$R = (\Sigma A)(\Sigma A \vee \Sigma D)^*$$

The keywords (for, while, if) are excluded in the lexical analysis phase following token recognition.

2. Design a DFA equivalent to R .

The DFA $M = \{Q, \Sigma, \delta, q_0, F\}$

$$Q = \{q_0, q_1, q_d\}$$

q_0 = Start state.

q_1 = accepting state (valid identifiers started)

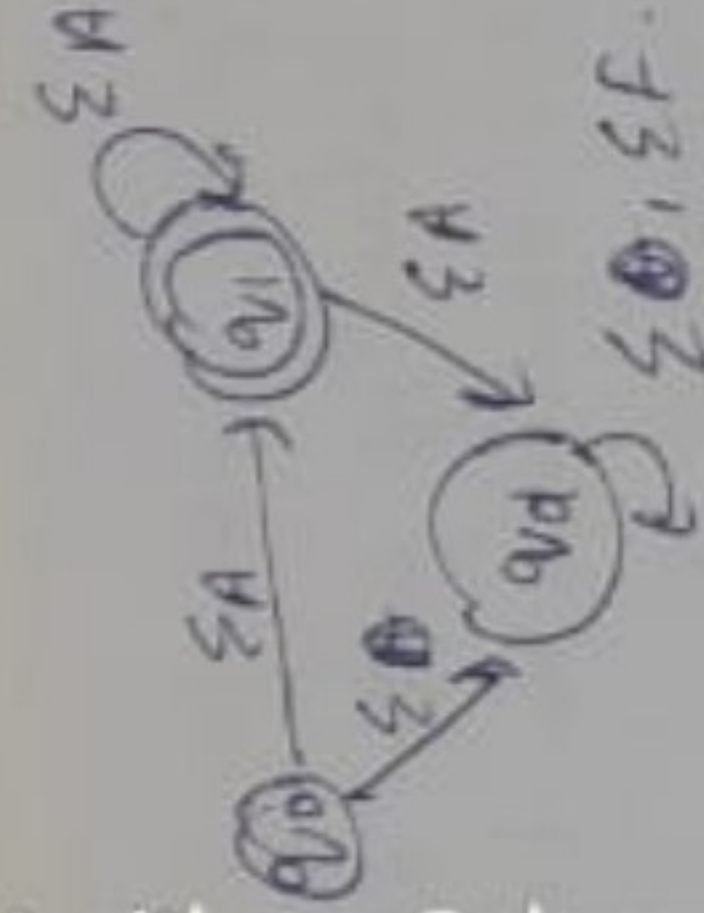
q_d = dead state (invalid start)

$$\Sigma = \Sigma A \vee \Sigma D$$

$$F = \{q_1\} \text{ (final state)}$$

Transition Table

State	Input ΣA	Input ΣD	Input (other)
q_0	q_1	q_d	q_d
q_1	q_1	q_1	q_1 (loop)
q_d	q_d	q_d	q_d



Embedding the DFA in a lexical analyser.

The DFA acts as the state machine for recognizing the
of an identifiers.

- 1) DFA recognition! The lexer consumes input characters against a small, finite list of reserved keywords (for, while, if). This is typically done via a fast hash table lookups
- 2) Keyword check! The recognized string is then checked input character, tracking DFA's state when the input stream forces the DFA out of q_1 (encountering a space or operator) the operator reader to that point is identified as a potential token.

3) Token generation!

- If the string is found in the keyword list, a keyword token is generated.
- otherwise, an identifier token is generated & its entry (lexical & type) is stored in the symbol table.

Unit - 2 PDA & Context free language.

Formulate a CFG for well formed queries.

Let $O = \{ \langle open \rangle \}$ & $C = \{ \langle close \rangle \}$. The grammar G modules balanced nesting.

$$S \rightarrow OSc|SS|E$$

$S \rightarrow OSc$ handles nested structure (eg $\langle open \rangle$

$S \rightarrow SS$ handles concatenated structure
..... $\langle close \rangle$

(eg <open> </close> <open>

$S \rightarrow \epsilon$ handles empty query.

2) Construct a PDA that accept such queries.

The PDA accepts the languages by empty stack. It uses the stack to track unmatched <open> tags.

$M = (\{q_0\}, \{0, c\}, \{z_0, x\}, \{q_0, \epsilon\})$

State	Input	Top of state	New state	Stack operation	Rational.
q_0	0	z_0	q_0	xz_0	Push x for first 0
q_0	0	x	q_0	xx	Push x for nested 0
q_0	c	x	q_0	E	Pop x for matching c
q_0	ϵ	z_0	q_0	E	Accept by empty stack.

3) Demonstrate the parse tree.

Query: <open> <open> </close> </close>

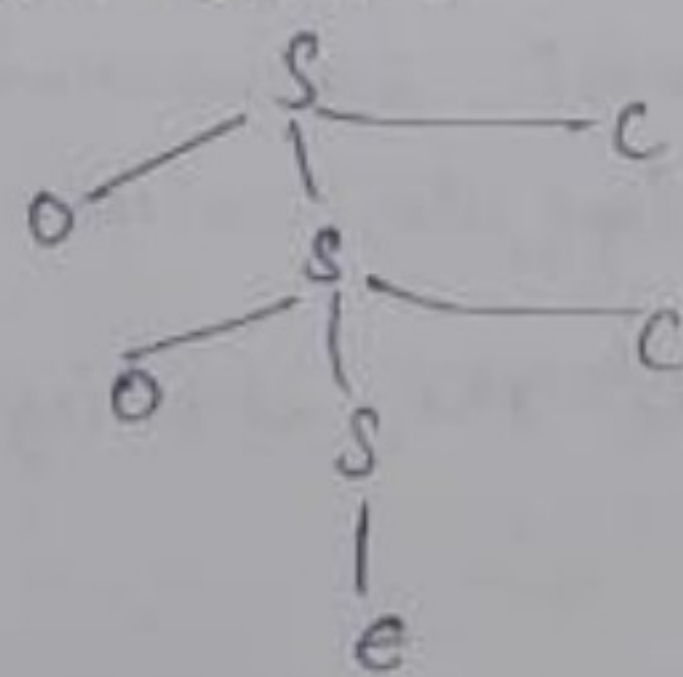
Derivation.

$S \Rightarrow 0Sc$

$S \Rightarrow (0Sc)c \quad (S \rightarrow 0Sc)$

$S \Rightarrow 0(0Ec)c \quad (S \rightarrow E)$

Parse tree:



Q-3 Unit-3 Turing Machine & Chomsky Hierarchy.

1) Justify why $L = \{a^n b^n c^n \mid n \geq 1\}$ is not Context free.
We use pumping lemma for CFL's.

1) Choose string s : Let p be the pumping length choose
 $s = a^p b^p c^p \in L$

2) Decompose s : $s = uvwxy$ where $|vwx| \leq p$ & $|vx| \geq 1$

3) Pumping argument since $|vwx| \leq p$ the pumpable segment vwx can only contain symbols from at most two blocks (only a's & b's & c's)

- Case (vwx is in a's & b's) pumping up (setting $i=2$) $\uparrow$ the no of a's and/or b's but leaves the number of c's fixed at p

- Resulting string $s' = uv^2wx^2y$ has unequal counts of a's, b's & c's (specifically, $\text{Count}(a) + \text{Count}(b) > \text{Count}(c)$)

4) Conclusion $s' \notin L$. Since the conditions of the pumping lemma are violated, L can't be a CFL.

1) Design a Turing Machine (TM) that accepts L .
The TM marks one a, one b & one c in the cycle until all symbols are marked.

- Tape alphabet $\Gamma: \{a, b, c, x, y, z, \square\}$ (x, y, z are markers)

- Core logic

q_0 : Mark the leftmost a as x & transition to find b

q_1 : Find the leftmost unmarked c & mark it as z
then transition to return

q_{next} : Soon left to the starting point (x)

qcheck: After all a's are marked, scan right to ensure the rest of the tape is just 4's, 2's & finally $\square$ (B)

step by step Configuration for 'aaa bbb'

The TM cycles three times to make the three pairs

- 1) Cycle 1 (Mark a₁ b₁ c₁): q₀, a a a b b b c c c → q₀, x a a b b c c c
 (Mark a) → q_b, x a a y b b c c (Mark b) → q_{int}, x a a y b b z c c
 (Mark c, return left) → q₀, x a a y b b z c c (Restart)
- 2) Cycle 2 (Mark a₂ b₂ c₂): q₀, x a a b b c c c → q_{int},
 x x a 4 4 b z z c (Tape becomes x x a 4 4 b z z c)
- 3) Cycle 3 (Mark a₃ b₂ c₃): q₀, x x a 4 4 b z z c → a ret,
 x x x 4 4 4 z z z.
- 4) Final check (qcheck): q₀ reads the marked a's (x's)
 , transitions to qcheck. qcheck scans over 4's & 2's
 until it hits $\square$. qcheck, x x x 4 4 4 z z z $\square$ → qaccept
 (accept)

(Q-4) Unit: 4 Code generation & optimization.

Expression $(A+B) * (C-D) + E$

- 1) Syntax - Directed Translation Scheme (S attributed)
 using a simple precedence-based grammar.

Production

$E \rightarrow E_1 + T$

$T \rightarrow T_1 * F$

Semantic Rules.

$E.addr = new_temp(); E.code =$
 $E_1.code || T.code || E.addr = E_1.addr +$
 $+ T.addr;$

$T.addr = new_temp(); T.code =$
 $T_1.code || F.code || T.addr = T_1.addr$
 $* F$
 $T.addr;$

$F \rightarrow (E1)$

$F \rightarrow Pd$

$F.addr = E1.addr; F.Code = E1.Code$

$F.addr = id.lexeme; F.Code = E$

- 2) Generate three address code (TAC)
The TAC is generated based on expression's evaluation order (precedence: parenthesis $\rightarrow$ multiplication $\rightarrow$ addition).

1) $t_1 = A + B$

2) $t_2 = C - D$

3) $t_3 = t_1 * t_2$

4) $t_4 = t_3 + E$

- 3) optimise the generated TAC

There is no duplicate expression (common subexpressions) in this lines 1 & 2. The code is already optimal work to CSE

Dead Code Removal.

Assume the final result t_4 is used, all intermediate variable (t_1, t_2, t_3) are necessary inputs for subsequent lines. No dead code can be removed.

optimised TAC (unchanged)

1) $t_1 = A + B$

2) $t_2 = C - D$

3) $t_3 = t_1 * t_2$

4) $t_4 = t_3 + E$

Q-5) (Cumulative - Advanced Reasoning & Application)

Language L = Equal no of 0's & 1's & no prefix has 1's than 0's (Dyck Paths)

prove that L is Context free but not regular
 not Regular: Use the pumping lemma for regular languages. Choose $s = 0^b 1^b$. pumping down ($i=0$) gives $0^{b-b} 1^b = 1^b$ ($b \geq 1$) which has unequal counts violating L . Thus L is not Regular which has unequal counts violating L . Thus L is not Regular.

- Context Free: The language L is accepted by a pushdown Automaton (PDA) shown below which demonstrates its Context free nature. The PDA's stack is essential for counting & comparing the non local dependencies (0s vs 1s)

Provide a CFG for this language (L)

The grammar G must enforce that every 1 is matched by a precedence 0

$$S \rightarrow 0S1 \mid \epsilon$$

This CFG generates all valid Dyck paths.

3) Design a PDA & Trace '0011'

A) PDA Design (M)

M accepts by empty stack, using x to count the excess no of 0's.

$$\text{Start } z_0: \delta(q_0, 0, z_0) = \{(q_0, xz_0)\}$$

$$\text{push } 0: \delta(q_0, 0, x) = \{(q_0, xx)\}$$

$$\text{pop } 1 (\text{prefix check}): \delta(q_0, 1, x) = \{(q_0, \epsilon)\}$$

(pop only if 0's are in excess)

Accept : $\delta(q_0, t, z_0) = \{q_0, \epsilon\}$

B) Trace the acceptance of '0011'

Input	State	Stack (L+R)	Transition	Condition check. (0's > 1's)
0011	q_0	z_0	push 0	
			push 0	$2 > 0$
0011	q_0	xz_0		
0011	q_0	xxz_0	pop 1	$2 > 1$
0011	q_0	xz_0	pop 1	$2 > 2$
0011	q_0	z_0	Empty stack	$2 = 2$
		ϵ	accept	